

Spring-2022

1 (a): Specify each phase of a compiler explicitly with corresponding input and output while processing the following statement:

Expression := Term * factor + 128

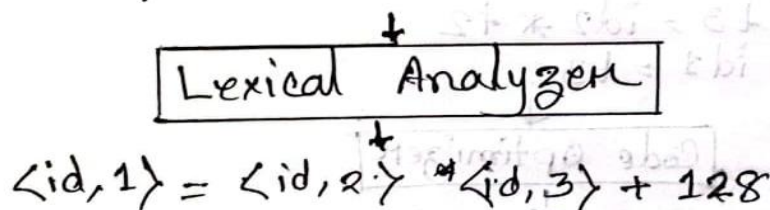
Solution:

⇒ There are different phases of compiler. The phases are, —

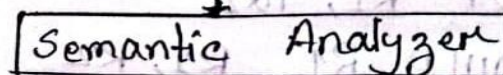
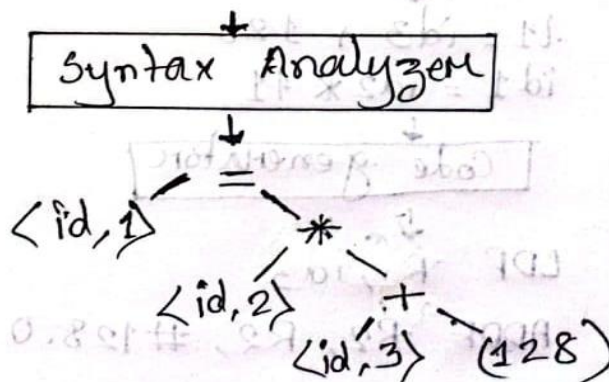
01. Lexical Analysis.
02. Syntax Analysis.
03. Semantic Analysis.
04. Intermediate Code generator.
05. Code Optimization.
06. Code Generation.

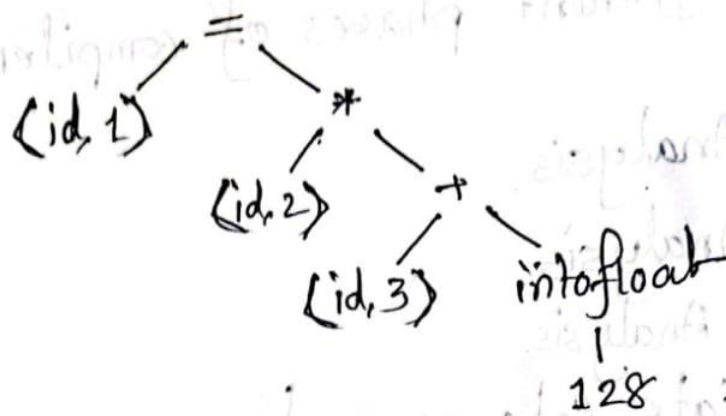
The given statement is —

Expression := Term * factor + 128



	token	name
1	id	Position
2	id	initial
3	id	rate





↓
Intermediate Code Generator

↓
 $t1 = \text{intofloat}(128)$

$t2 = id3 + t1$

$t3 = id2 * t2$

$id1 = t3$

↓
Code Optimizer

↓
 $t1 = id3 + 128$

$id1 = id2 * t1$

↓
Code generator

↓
 LDR R2, id3

ADDF R2, R2, #128.0

LDR R1, id2

MULR R1, R1, R2

STF id1, R1

1 (b): Consider the grammar:

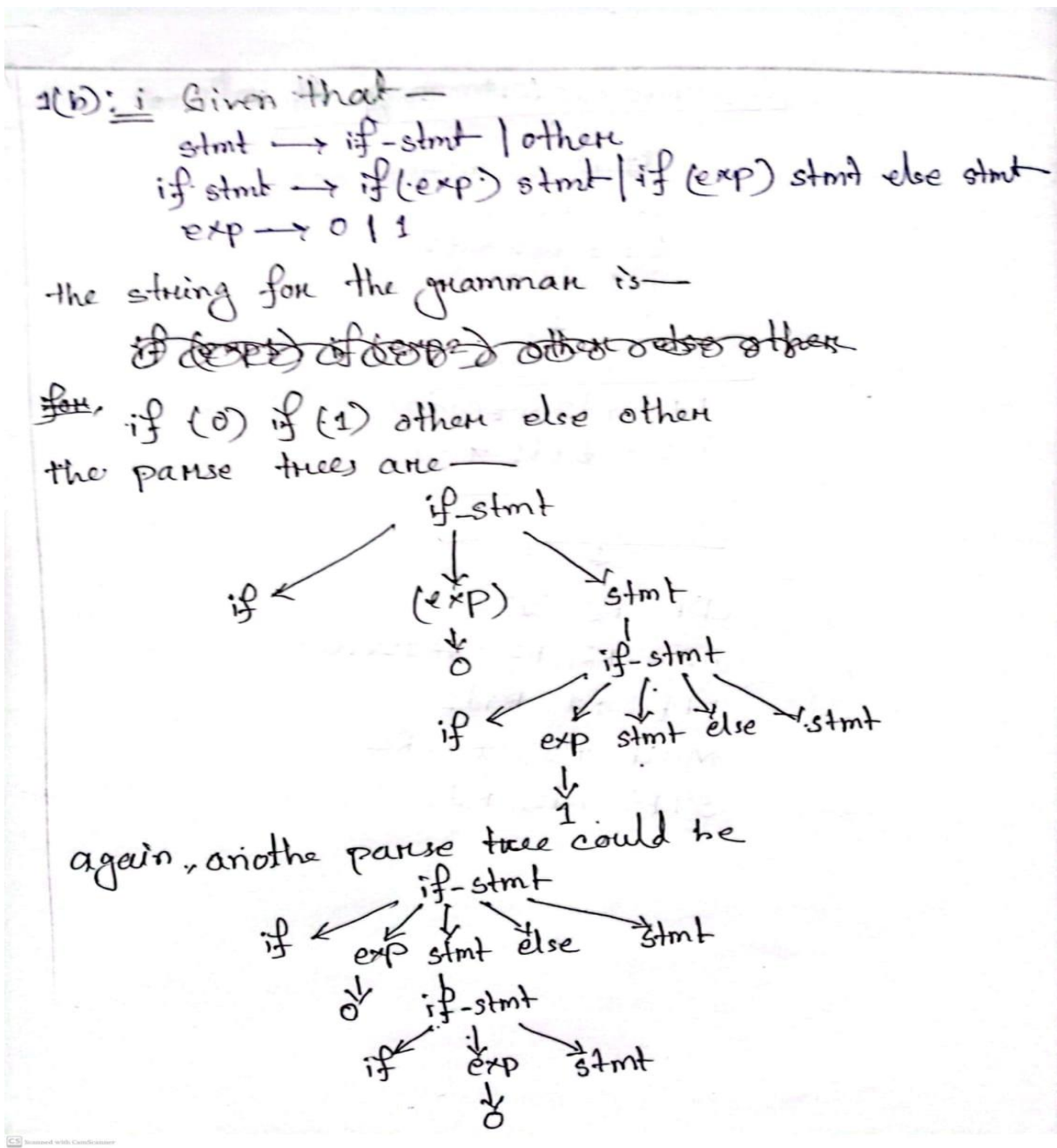
$\text{Stmt} \rightarrow \text{if-stmt} \mid \text{other}$

$\text{If-stmt} \rightarrow \text{if}(\text{exp}) \text{ stmt} \mid \text{if}(\text{exp}) \text{ stmt else stmt}$

$\text{Exp} \rightarrow 0 \mid 1$

- Now, show that the grammar is ambiguous by constructing two parse trees for the following string:
If (0) if (1) other else other
- Rewrite the grammar so that it becomes unambiguous.

Solution(i):



Solution(ii):

1(b): ii : Eliminating the ambiguity -

$stmt \rightarrow if-stmt \mid other$

$if-stmt \rightarrow if (exp) stmt \mid if (exp) ~~stmt~~ full-stmt else stmt$

$full-stmt \rightarrow full-if \mid other$

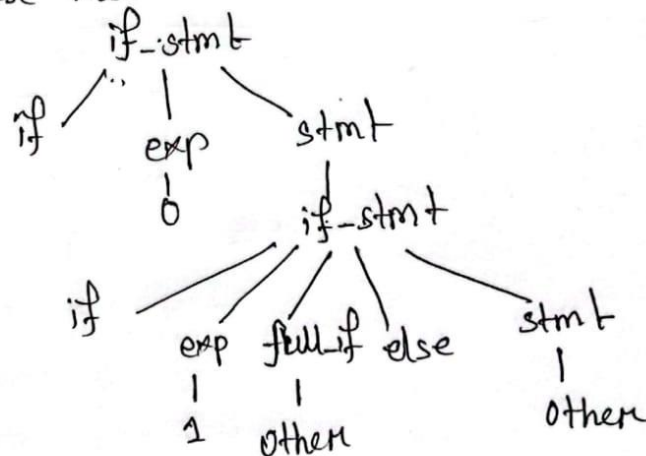
$full-if \rightarrow if (exp) fullstmt else fullstmt$

$exp \rightarrow 0 \mid 1$

the string for the grammar is -

$if (0) if (1) other else other$

the correct parse tree is -



2(a): Consider the following grammar:

$E \rightarrow E \text{ or } T \mid T$

$T \rightarrow T \text{ and } F \mid F$

$F \rightarrow \text{not } F \mid (E) \mid \text{true} \mid \text{false}$

Now, construct a parse tree for the following sequence:

Not (true or false)

Solution:

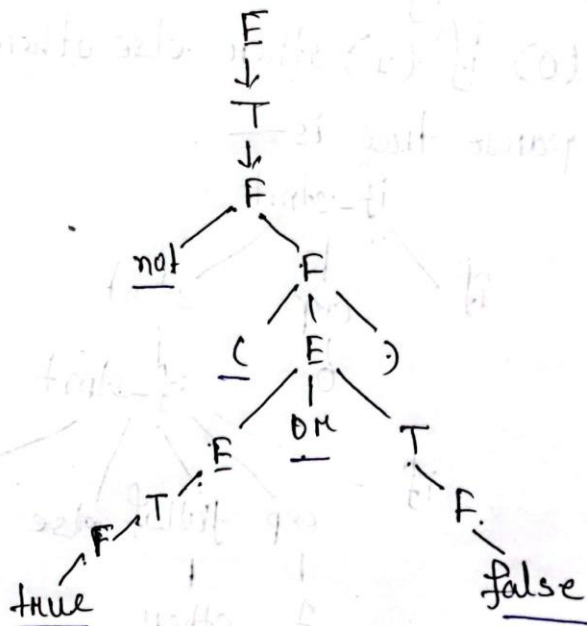
2(a): Given that,

$E \rightarrow E \text{ or } T \mid T$

$T \rightarrow T \text{ and } F \mid F$

$F \rightarrow \text{not } F \mid (E) \mid \text{true} \mid \text{false}$

and the sequence is not (true or false).
the parse tree is —



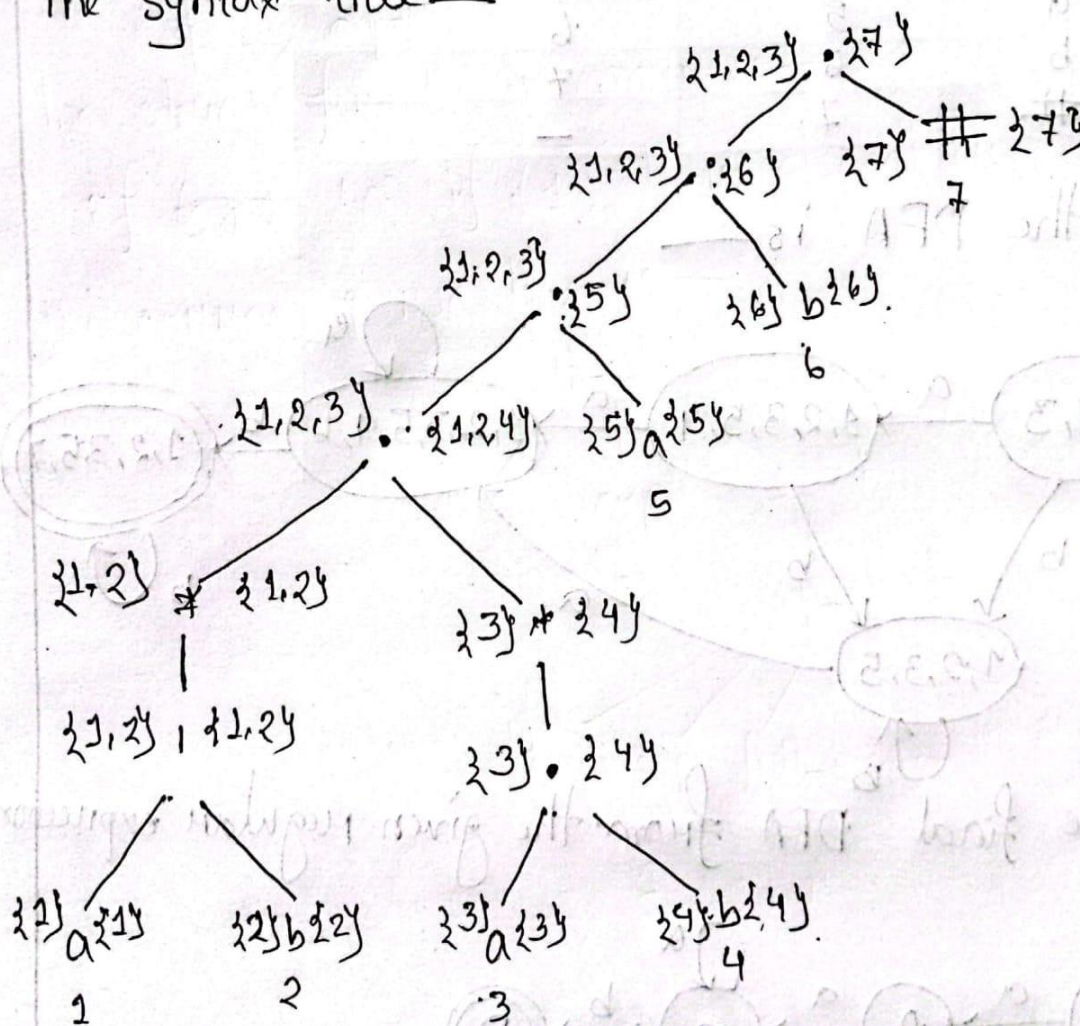
This is the parse tree for the given sequence.

2(b): Suppose you are considering DFA from regular expression. Now, compute the following positions for the following regular expression:

$(a|b)^*ab^*ab$

Solution: (Not really sure about it. It's recommended to check it yourself once.)

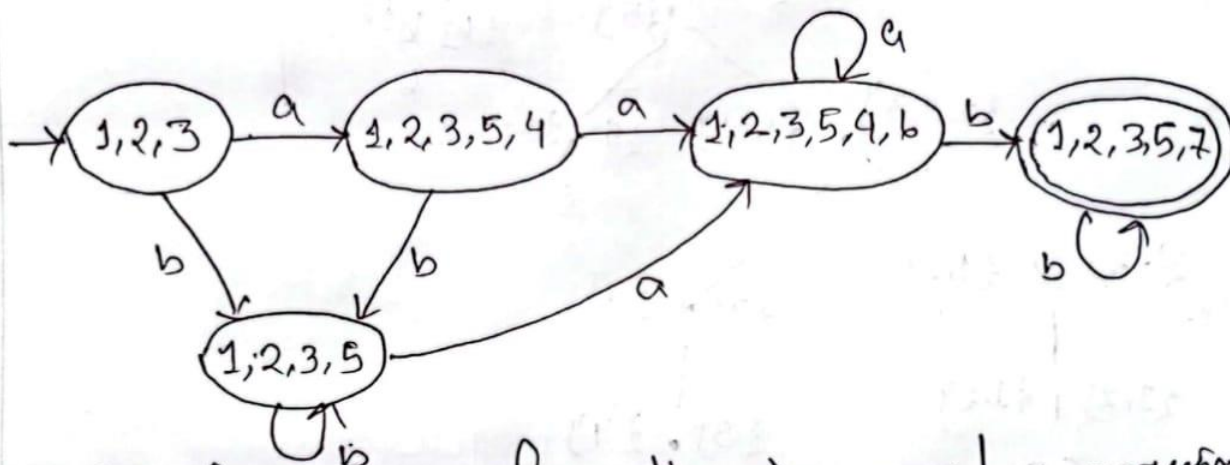
2(b) Given that
 regular expression is $(a|b)^*ab^*ab$
 By concatenating a unique right end marker '#'
 regular expression is $(a|b)^*ab^*ab\#$
 The syntax tree —



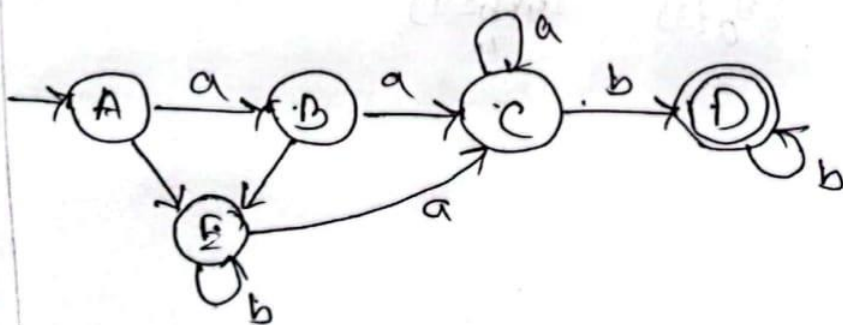
Now computing the followposition —

Symbols	nodes (n)	followposition
a	1	{1, 2, 3, 5}
b	2	{1, 2, 3, 5}
a	3	4
b	4	3, 5
a	5	6
b	6	7
#	7	—

now the DFA is —



So, the final DFA from the given regular expression —



OR

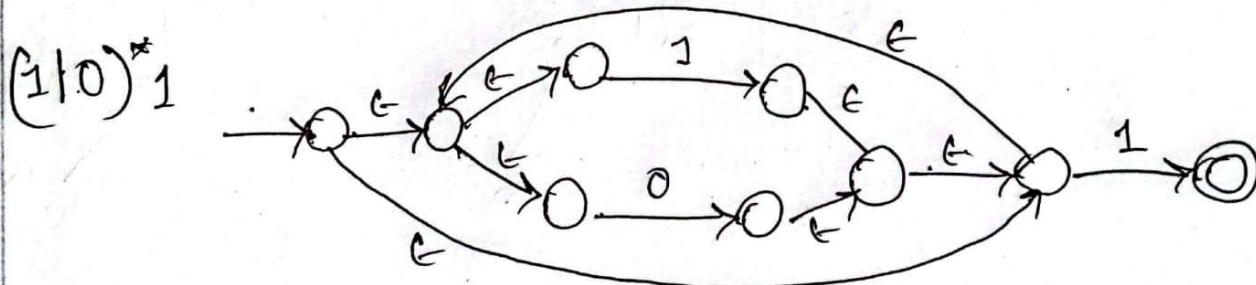
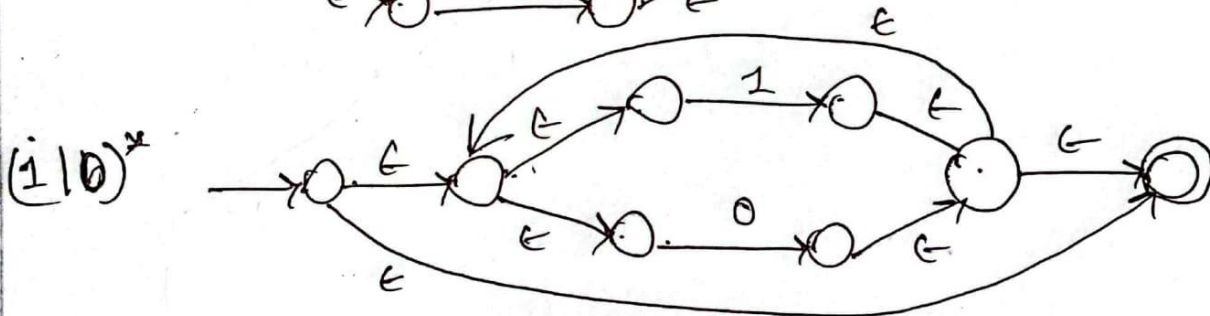
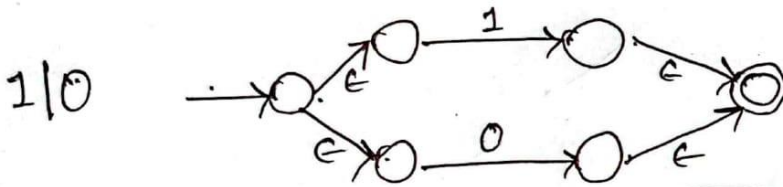
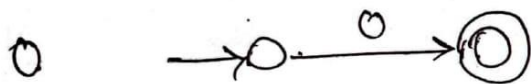
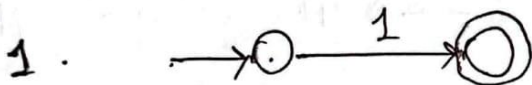
2(b): Convert the regular expression $R = (1|0)^*1$ to its equivalent NFA using Thompson construction.

Solution:

2(b): Given that,

Regular expression $R = (1|0)^*1$.

Converting R to equivalent NFA using Thompson construction.



3(a):

- i. What are the advantages of separating the analysis phase of compiling into lexical analysis and parsing?
- ii. Why context free grammar is preferred over regular expressions in parsing?

Solution(i):

Lexical analysis separated from analysis phase and parsing because of following advantages

Simpler design.

- Separation allows the simplification of one or the other.
- Example: A parser with comments or white spaces is more compel

Compiler efficiency is improved.

- Optimization of lexical analysis because a large amount of time is spent reading the source program and partitioning it into tokens.

Compiler portability is enhanced.

- Input alphabet peculiarities and other device-specific anomalies can be restricted to the lexical analyzer.

Solution(i):

context free grammar is preferred over regular expressions in parsing because:

Regular Expressions are most useful for describing the structure of lexical construct such as identifiers, constant etc. In context free grammar, There is no specific guideline for lexical and syntactic analysis in case of Context free grammar. Context free grammars are most useful in describing the nested chain structure or syntactic structure such as balanced parenthesis, if else etc.and these can't be define by Regular Expression.

3(b): Convert the regular expression $R=(a|b)^*abb(a|b)$ directly to its equivalent DFA.

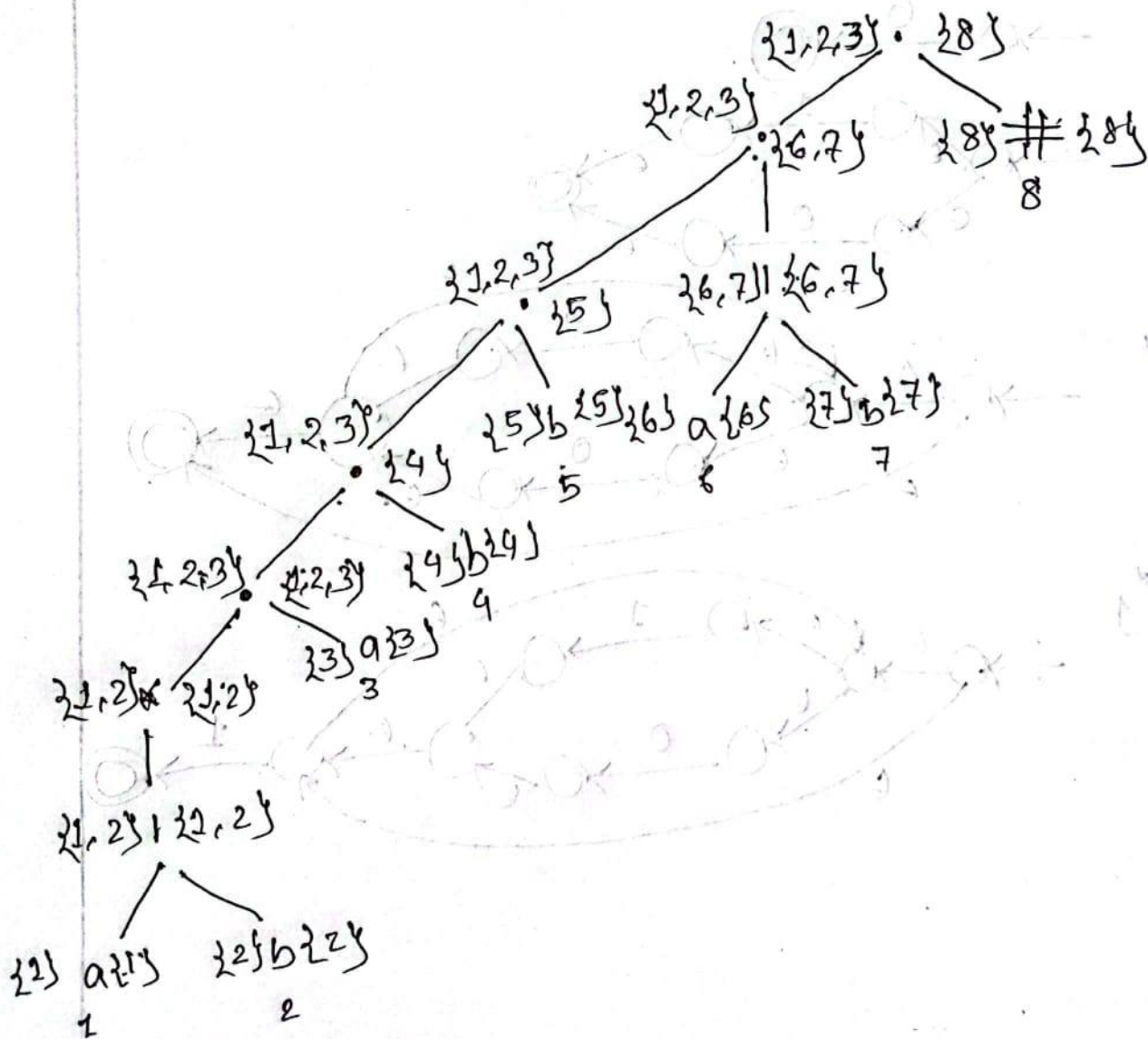
Solution: (Not really sure about it. It's recommended to check it yourself once.)

3(b): Given that,

$(a|b)^*abb(a|b)$

By concatenating a unique right end marker
 $(a|b)^*abb(a|b)\#$

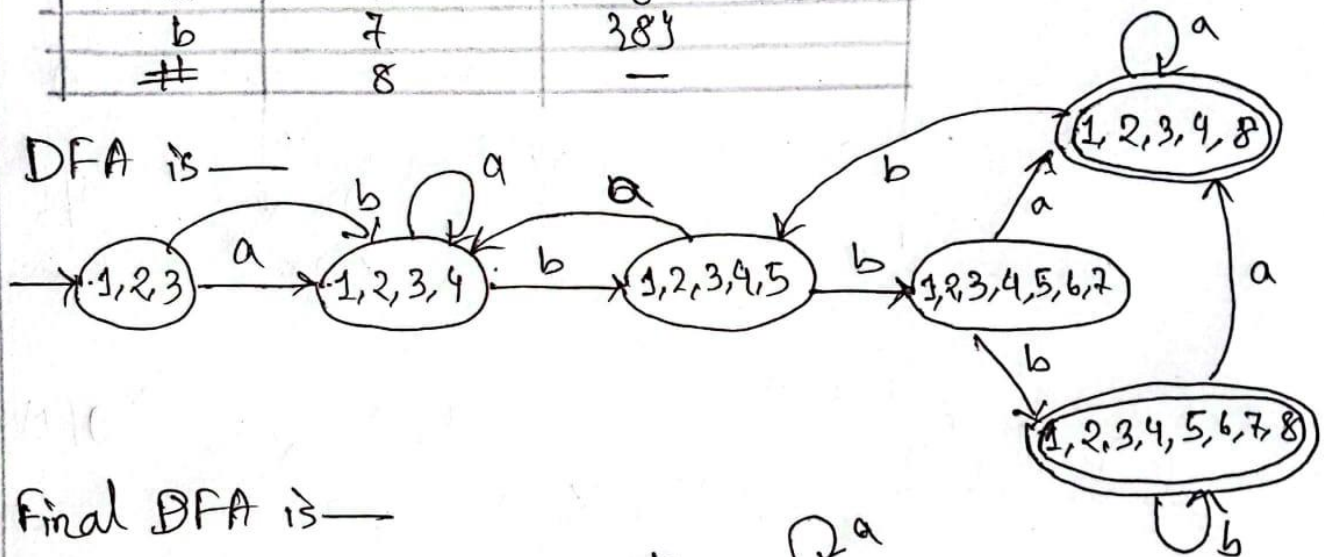
The syntax tree —



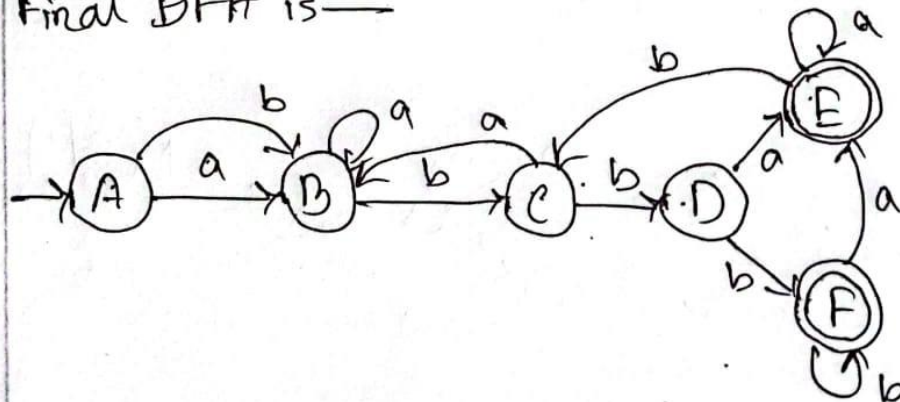
Now computing followposition —

symbol	nodes (n)	followposition
a	1	{1, 2, 3, 4}
b	2	{1, 2, 3, 4}
a	3	{4}
b	4	{5}
b	5	{6, 7}
a	6	{8}
b	7	{8}
#	8	—

DFA is —



Final DFA is —



Note: For understanding RE to DFA direct conversion you can check these videos serially for better understanding.

01. <https://youtu.be/7iKXO6cZjWg?si=Z8eBuk6NQEst2wv>

02. <https://youtu.be/HgFYPLmnMbs?si=cbh9peffovndn0n1>

03. https://youtu.be/XU_7DDNfd4M?si=sYmzcOf7ad9rlib3